

ASP.NET Core Web API ve HTTP

Console uygulamasından istemci-sunucu dünyasına geç. İlk çalışan endpoint'i oluştur, isteği gönder ve cevabı oku.

Yaklaşık	Odak	Çıktı
150–210 dk	HTTP akışı	Çalışan ilk API
Araç	Şablon	Test
.NET CLI	ASP.NET Core	Tarayıcı / curl

Bugünün çalışma kuralı

Önce API'yi çalıştır ve tek endpoint'i test et. Controller, katmanlar ve veritabanını aynı günün ilk örneğine ekleme.

Bitiş ölçütü

- İstemci, sunucu, API ve endpoint kavramlarını açıklayabilmek
- HTTP isteği ile cevabın parçalarını ayırt edebilmek
- GET isteği ve temel durum kodlarını tanımak
- JSON döndüren iki minimal endpoint yazıp test etmek

1. Web API nedir?

Bir Web API, farklı uygulamaların HTTP üzerinden veri ve işlem paylaşmasını sağlar. Arayüz şart değildir; istemci API'ye istek gönderir, API cevap verir.

İstemci	HTTP	Sunucu / API
Tarayıcı, mobil uygulama, başka servis	İstek ve cevap kuralları	İsteği işler ve cevap üretir

Temel akış

İSTEMCİ -- HTTP İSTEĞİ --> API

İSTEMCİ <-- HTTP CEVABI -- API

Günlük örnek

Restoranda müşteri istemci, garson iletişim kanalı, mutfak sunucu gibidir. Sipariş istek; hazırlanan yemek cevaptır.

API'nin görevi

- İsteğin ne istediğini anlamak
- Gerekli işlemi yapmak
- Uygun durum kodunu seçmek
- Çoğunlukla JSON biçiminde cevap döndürmek

2. HTTP isteği ve cevabı

HTTP iletişimi iki ana parçadan oluşur: request (istek) ve response (cevap).

Örnek istek

```
GET /api/products HTTP/1.1
Host: localhost:5000
Accept: application/json
```

Örnek cevap

```
HTTP/1.1 200 OK
Content-Type: application/json

[
  { "id": 1, "name": "Klavye", "price": 850 }
]
```

Parça	Anlamı
Method	Ne yapılacağını belirtir: GET, POST...
Path	Hangi kaynağa gidildiğini belirtir.
Headers	İstek/cevap hakkında ek bilgi taşır.
Body	Gönderilen veya dönen veriyi taşır.
Status code	İşlemin sonucunu özetler.

Endpoint

HTTP metodu ile adresin birleşimidir: GET /api/products. Aynı adres farklı metotlarla farklı işler yapılabilir.

3. HTTP metotları ve durum kodları

Bugün GET ile çalışan ilk endpoint'i yazacağız; diğer metotların amacını şimdiden tanıyacağız.

Metot	Genel amaç	Örnek
GET	Veri okumak	Ürünleri getir
POST	Yeni veri oluşturmak	Ürün ekle
PUT	Kaynağı bütünüyle güncellemek	Ürünü değiştir
PATCH	Kısmi güncelleme	Yalnızca fiyatı değiştir
DELETE	Kaynağı silmek	Ürünü sil

Sık durum kodları

Kod	Ad	Anlam
200	OK	İstek başarıyla işlendi.
201	Created	Yeni kaynak oluşturuldu.
400	Bad Request	İstek veya veri geçersiz.
404	Not Found	İstenen kaynak bulunamadı.
500	Server Error	Sunucuda beklenmeyen hata.

Kategori mantığı

2xx başarı, 4xx istemci kaynaklı sorun, 5xx sunucu kaynaklı sorun. Durum kodu cevabın kısa özetidir.

4. JSON: API'nin ortak veri dili

JSON, veriyi anahtar-değer yapısında taşıyan metin biçimidir. C# nesnesi cevap olarak döndürüldüğünde ASP.NET Core çoğunlukla onu otomatik JSON'a çevirir.

C# nesnesi

```
var product = new
{
    Id = 1,
    Name = "Klavye",
    Price = 850,
    InStock = true
};
```

HTTP cevabındaki JSON

```
{
  "id": 1,
  "name": "Klavye",
  "price": 850,
  "inStock": true
}
```

JSON	C# karşılığı
"Klavye"	string
850	int / decimal
true	bool
[...]	Dizi / List
{ ... }	Nesne

Dikkat

JSON property adlarında çift tırnak bulunur. Metin değerleri de çift tırnak içindedir; sayı ve bool değerleri değildir.

5. İlk ASP.NET Core projesi

Proje klasörünü .NET şablonuyla oluşturun. Şablon SDK sürümüne göre küçük farklılıklar gösterebilir; temel çalışma akışı aynıdır.

```
cd "C:\Users\Administrator\.NetEgitim"
mkdir DotNetIkinciHafta
cd DotNetIkinciHafta

dotnet new webapi -n Gun8-WebApi
cd Gun8-WebApi
code .
```

Temel dosyalar

Dosya	Görevi
Program.cs	Uygulamanın kurulumu ve endpoint'ler
*.csproj	SDK, hedef framework ve paket bilgileri
appsettings.json	Uygulama ayarları
Properties/launchSettings.json	Yerel çalıştırma adresleri

Çalıştır

```
dotnet run
```

Terminalde "Now listening on" satırındaki adresi kullan. Port numarası bilgisayara göre değişebilir:

```
Now listening on: http://localhost:5000
```

Port nedir?

Aynı bilgisayardaki farklı uygulamaları ayıran kapı numarasıdır. Kodunda sabit bir port varsaymak yerine terminaldeki adresi takip et.

6. İlk çalışan endpoint

Program.cs içeriğini sadeleştirerek ilk GET endpoint'ini kurabiliriz.

```
var builder = WebApplication.CreateBuilder(args);  
var app = builder.Build();  
  
app.MapGet("/", () => "API çalışıyor");  
  
app.Run();
```

Satırların görevi

Satır	Anlamı
CreateBuilder	Uygulama yapılandırmasını hazırlar.
Build	Web uygulamasını oluşturur.
MapGet	GET isteğini bir adrese bağlar.
Run	Sunucuyu başlatıp istekleri dinler.

Test

```
dotnet run  
  
# Terminaldeki adres örneğin http://localhost:5000 ise:  
curl http://localhost:5000/
```

Beklenen cevap:

```
API çalışıyor
```

Program neden kapanmıyor?

Console projesinden farklı olarak web sunucusu sürekli istek bekler. Durdurmak için terminalde Ctrl + C kullan.

7. İlk görev — JSON endpoint

Çalışan metin endpoint'inden sonra ikinci endpoint'i ekle. Tam çözüm öğrenci kodu görülmeden verilmez.

Görev

- Adres GET /api/info olsun.
- Cevap anonim bir nesne olsun.
- Cevapta application, day ve isActive alanları bulunsun.
- API'yi çalıştırıp tarayıcı veya curl ile test et.

İskelet

```
app.MapGet("/api/info", () =>
{
    return new
    {
        Application = /* uygulama adı */,
        Day = /* gün numarası */,
        IsActive = /* bool değer */
    };
});
```

Beklenen JSON biçimi

```
{
  "application": "DotNet Eğitimi API",
  "day": 8,
  "isActive": true
}
```

Sıra önemli

MapGet çağrılarını app.Run() satırından önce olmalı. Run sunucunun çalışma döngüsünü başlatır.

Bitiş testi

- GET / metin döndürüyor.
- GET /api/info JSON döndürüyor.
- Terminalde istek satırları görülüyor.
- Ctrl+C sunucuyu güvenli biçimde durduruyor.

Git

```
cd "C:\Users\Administrator\.NetEgitim"
git add .
git commit -m "Gün 8 Web API ve HTTP temelleri"
git push
```